

De los datos a las respuestas: modelando, consultando y protegiendo el backend

Datos de la actividad

Número actividad	2
Docente	David Aldón Mínguez
Lengua de docencia	Castellano
Agrupación	Individual
Fecha de entrega	04-05-2026

Descripción de la actividad

La Actividad 2 introduce al estudiante en los fundamentos del desarrollo backend y el acceso a datos dentro del contexto de aplicaciones web modernas. El objetivo es comprender cómo se modelan datos en una base de datos relacional y cómo se diseñan APIs que respondan a preguntas de negocio reales, aplicando patrones de acceso a datos y mecanismos básicos de seguridad habituales en entornos profesionales.

A partir del caso ficticio de DataSphere, el estudiante trabaja con un modelo simplificado que representa activos geolocalizados, categorías, tipos de peligros naturales y niveles de exposición asociados. Con este modelo diseña endpoints que permiten responder consultas funcionales relacionadas con la exposición al riesgo —como agregaciones, filtrados y búsquedas por zona— aplicando de forma ligera patrones como Repository y Unit of Work, junto con la elaboración de tests unitarios simples que refuercen la verificación del comportamiento esperado.

La actividad incorpora también la aplicación introductoria de autenticación y autorización mediante JWT y roles, simulando cómo se protege el acceso a información sensible dentro de un servicio backend. Todas las tareas pueden resolverse mediante implementaciones completas o mediante versiones simplificadas (dummy), garantizando que el aprendizaje conceptual sea accesible para todo el alumnado.

Finalmente, el estudiante utiliza SQL Server Express/LocalDB, Postman, Visual Studio y Git para modelar datos, probar endpoints y documentar su progreso con buenas prácticas de versionado. En conjunto, la actividad acerca al estudiante a un entorno de trabajo próximo a la realidad, integrando base de datos, API y seguridad como componentes esenciales de una aplicación web moderna.

Competencias

COMPETENCIAS (CE / CT)	RESULTADO DE APRENDIZAJE (RA)	INDICADORES DE EVALUACIÓN (IE)
CE2	RA1. Resumir las diferentes tecnologías web y sus interacciones.	IE1. El estudiante identifica las tecnologías implicadas en una aplicación web backend y describe su interacción (API, base de datos y clientes).



COMPETENCIAS (CE / CT)	RESULTADO DE APRENDIZAJE (RA)	INDICADORES DE EVALUACIÓN (IE)
CE2	RA2. Interpretar el paradigma cliente–servidor utilizado en las tecnologías web.	IE2. El estudiante explica cómo una API responde a peticiones cliente–servidor y lo relaciona con la arquitectura backend.
CE2, CT3	RA3. Utilizar las técnicas y los procedimientos más adecuados para el diseño de una aplicación web a partir de diferentes fuentes de información.	IE3. El estudiante diseña endpoints y estructuras de acceso a datos coherentes, justificando las decisiones técnicas a partir de documentación.
CE2	RA4. Analizar la idoneidad de las diferentes tecnologías web y su integración según las necesidades del usuario.	IE4. El estudiante argumenta la elección de tecnologías backend (API, base de datos, patrones) en función de los requisitos funcionales.
CE2	RA6. Diseñar un sitio web que integre la interfaz de cliente, servidor y base de datos.	IE6. El estudiante integra correctamente la API con una base de datos o una fuente de datos simulada.
CE3	RA7. Analizar y evaluar información desde una amplia gama de fuentes digitales utilizada para dar solución a un problema.	IE7. El estudiante valida el comportamiento de la API mediante pruebas básicas (tests unitarios y/o pruebas funcionales con Postman).

Contenido de la actividad

Contenidos	ODS
Diseño e implementación de APIs REST para responder a consultas funcionales.	ODS 4 – Educación de calidad El alumnado desarrolla competencias digitales avanzadas mediante la implementación práctica de servicios web.
Modelado y acceso a datos mediante bases de datos relacionales y operaciones CRUD.	ODS 9 – Industria, innovación e infraestructura Se fomenta el diseño de infraestructuras digitales modernas y estructuradas.
Aplicación de patrones arquitectónicos básicos (Repository Pattern y Unit of Work).	ODS 9 – Industria, innovación e infraestructura Se promueve la innovación a través de arquitecturas backend mantenibles y reutilizables.
Uso de herramientas profesionales para validación y pruebas de servicios web.	ODS 8 – Trabajo decente y crecimiento económico La actividad aproxima al estudiante a prácticas y herramientas habituales del entorno profesional.

Descripción de las tareas

Construyendo la API de Datos de Riesgo: Backend, SQL y Seguridad Aplicada Tu primera funcionalidad real dentro de DataSphere Technologies

✉ Correo de Ingrid (Lead Architect) — 08:11 AM



Asunto: *Siguiente paso: crear la API que permitirá analizar riesgos naturales*

Hola,

Ahora que ya comprendes la arquitectura general de DataSphere, es momento de trabajar con uno de los componentes clave del sistema: **la API interna**. Esta API es la encargada de responder preguntas reales de negocio, como:

- ¿Cuántos activos están expuestos en esta zona?
- ¿Cuál es el valor económico total de los activos afectados?
- ¿Qué categoría de activo es más vulnerable ante cierto fenómeno?

! Importante

Esta actividad **no depende de la Actividad 1**.

Puedes completarla incluso si no pudiste realizar la anterior.

Además, **cada tarea dentro de esta actividad es independiente**.

He programado reuniones con algunos compañeros para ayudarte.

Nos vemos pronto,

— Ingrid

 Herramientas necesarias para completar la actividad

1. Visual Studio / Visual Studio Code

- Para crear la API mínima (.NET 7/8).
- Puedes usar plantillas de Minimal API o controlador clásico.

2. SQL Server Express / LocalDB

- Uso: base de datos local con una tabla Assets.
- Descarga: <https://www.microsoft.com/sql-server/sql-server-downloads>

3. Postman

- Para probar endpoints o, en caso de versión dummy, simular las peticiones.
- Descarga: <https://www.postman.com/downloads/>

4. Git

- Commit atómico, estructura limpia y capturas del historial.
- Descarga: <https://git-scm.com/>

 1. Reunión con Viktor Novak — Senior Backend Engineer

Tema: Datos que representan activos expuestos en DataSphere

Viktor te explica:

En DataSphere trabajamos con **activos físicos distribuidos por el territorio** (edificios, instalaciones, infraestructuras, etc.) y con **distintos tipos de peligros naturales** (hazards) que pueden afectarlos: tornados, volcanes, huracanes, tormentas de lluvia, viento extremo, terremotos...

Cada activo tiene unas propiedades mínimas para que el sistema pueda calcular su **exposición al riesgo**:

Tabla Assets (activos)

- Id
- Latitude, Longitude
- CategoryId (referencia a la categoría del activo)
- BaseValue (valor económico asociado al activo)

Tabla Categories (tipos de activo)

- Id
- Name (industrial, residencial, agrícola, etc.)

Además, DataSphere necesita modelar **cómo cada activo se ve afectado por cada tipo de peligro**, ya que la exposición puede variar según el fenómeno:

Tabla Hazards (peligros naturales)

- Id
- Name (Tornado, Volcano, Hurricane, Rainstorm, Windstorm, Earthquake, ...)



Tabla AssetHazardExposure (exposición por activo y peligro)

- Id
- AssetId (referencia a Assets)
- HazardId (referencia a Hazards)
- ExposureValue (valor expuesto para ese activo ante ese peligro concreto)

También podemos registrar **condiciones adicionales** que modifican la vulnerabilidad de un activo:

Tabla Conditions (condiciones o factores)

- Id
- Name (por ejemplo: “cerca de la costa”, “estructura antigua”, “zona inundable”)

“Con este modelo relacional”, dice Viktor, “los analistas pueden construir métricas como:

- valor total expuesto ante un huracán en una zona concreta,
- qué categoría de activo es más vulnerable ante terremotos,
- cómo cambia la exposición si añadimos ciertas condiciones.”

■ TAREA 1 — Modelado SQL

Diseña e implementa un modelo relacional en una base de datos local que permita a DataSphere calcular la exposición al riesgo de sus activos frente a distintos peligros naturales.

El modelo debe incluir al menos **4–5 tablas** (como las que Viktor ha descrito: activos, categorías, peligros y la relación entre activos y peligros, más condiciones si lo consideras necesario), de forma que sea posible responder preguntas como:

- “¿Cuál es el valor total expuesto ante un tipo de peligro concreto?”
- “¿Qué activos están más expuestos a tornados en una región determinada?”

Inserta varios registros de ejemplo que tengan sentido (diferentes categorías, distintos peligros y valores de exposición realistas), de modo que el esquema no sea solo teórico, sino utilizable para consultas posteriores.

Entregables:

- Captura del modelo (tablas y relaciones) y del script ejecutado
- Captura de los registros de ejemplo en las tablas clave (Assets, Hazards, AssetHazardExposure, etc.)
- Tu nombre visible en las capturas

Frase sugerida:

“Este modelo relacional representa cómo DataSphere calcula la exposición de distintos activos frente a varios peligros naturales.”

Nota: Es importante justificar cada una de las decisiones tomadas en la creación de las tablas. ¿Por ejemplo, porque un decimal vs float? ¿Por qué una tabla de relación? Índices, claves etc

■ TAREA 2 — Construcción de la API CRUD de activos (con Repository Pattern, UoW y tests sencillos)

Construir una **API REST mínima** que permita gestionar activos del sistema (CRUD) y responder una consulta agregada de exposición. El diseño debe reflejar el uso profesional de una capa de acceso a datos mediante **Repository Pattern + Unit of Work**, y debe incluir **1–2 tests unitarios** sencillos.

Qué debes implementar (CRUD mínimo)

Implementa estos endpoints (o sus versiones dummy):

CRUD sobre activos

- **POST** /assets → crear un activo
- **GET** /assets/{id} → obtener un activo por id
- **PUT** /assets/{id} → actualizar un activo
- **DELETE** /assets/{id} → eliminar un activo

Consulta funcional (no CRUD)

- **GET** /assets/total-exposure → devuelve el total agregado del valor económico (por ejemplo suma de AssetValue)



Nota: Puedes usar **SQL Server/LocalDB** o **datos en memoria**. Ambos son válidos para esta tarea (no depende de la Tarea 1).

✓ **Estructura mínima requerida (ligera)**

- IAssetRepository con métodos CRUD + GetTotalExposure()
- UnitOfWork que exponga el repositorio
- Endpoints consumiendo el repositorio desde la UoW
- **Algunos tests unitarios** (xUnit o NUnit) sobre:
 - crear/leer un activo (aunque sea con datos dummy), y/o
 - GetTotalExposure() devolviendo un valor esperado.

■ **Si tu API es funcional (implementación completa)**

- Endpoints operativos (con SQL o memoria).
- Pruebas con Postman mostrando al menos:
 - 2 operación CRUD (por ejemplo POST + GET)
 - el endpoint total-exposure

Nota: ¡Si hay algo que no te sale, nunca dejes en blanco la tarea! Todo suma, si en algún punto te atascas, por ejemplo un controller o un pattern. Simplemente devuelve un return “demo” etc, y continua, ¡no te dejes puntos por el camino por que no salga algo!

📁 **Entregable**

Incluye en tu PDF capturas de:

1. **Código de los endpoints CRUD + total-exposure** (con tu nombre visible)
2. **Repository + Unit of Work** (con tu nombre visible)
3. **Al menos 2 tests** ejecutados (Test Explorer)
4. **Postman** mostrando:
 - una operación CRUD (ej. POST y luego GET)
 - y una llamada a GET /assets/total-exposure
(si tu API no corre, sustituye Postman por una captura del código con comentario “simulación”)

📝 Frase sugerida:

“Este CRUD permite gestionar activos y calcular métricas básicas de exposición en DataSphere.”

■ **3. Email de Samir — Security Engineer**

Tema: Proteger los endpoints sensibles

Samir insiste en que los valores económicos son sensibles y deben protegerse con:

- **JWT** para usuarios analistas
- **Roles** para delimitar accesos
- **API Keys** para procesos internos (aunque aquí solo lo representaremos)

■ **TAREA 3 — Seguridad aplicada**

Aplica autenticación y autorización a uno o más endpoints críticos, simulando la protección de información sensible dentro de DataSphere.

👉 **Si tienes API funcional:**

- Protege endpoint con [Authorize] y rol (ej. RiskAnalyst)
- Demuestra con Postman petición denegada → petición permitida

👉 **Si NO tienes API funcional:**

- Crea un método dummy, por ejemplo GetTotalExposureDemo()
- Añade atributos de autorización, ej.:
[Authorize(Roles = "RiskAnalyst")]
- Devuelve return Ok("demo protected data")



Entregables:

- Postman con error 401 + Postman con éxito, ambos con método decorado con atributos en la misma captura de pantalla
- Tu nombre visible



Frase:

“Este endpoint requiere autorización porque expone datos sensibles del análisis de riesgos.”

Nota: El enunciado es corto, pero pensad si os dejáis algo, si incluiríais algo más en las validaciones

TAREA 4 — Git Strategy corporativa y ciclo de despliegue

Reunión con Hiro — Senior Developer

Hiro te explica:

“En DataSphere no trabajamos directamente sobre main.

Somos varios equipos, entregamos cambios de forma continua y **desplegamos a producción de manera mensual**.

Antes de tocar código, quiero ver si sabes **cómo organizarías el trabajo del equipo con Git**.”

Objetivo

Diseñar una **estrategia de ramas en Git** que permita a varios desarrolladores trabajar en paralelo, integrar cambios de forma controlada y realizar **despliegues mensuales** de la aplicación.

No se espera código ni repositorio real.

Lo importante es **entender y representar el flujo de trabajo profesional**.

✓ **Qué debes hacer**

1 Diagrama de la estrategia Git (obligatorio)

Crea un **diagrama gráfico** (Draw.io / diagrams.net) que represente:

- Rama main (producción)
- Rama dev (integración continua)
- Ramas de trabajo:
 - feature/*
 - bugfix/*
- Flujo de merges entre ramas
- Momento en el que se produce el **despliegue mensual** desde main

El diagrama debe mostrar claramente:

- dónde trabajan los desarrolladores,
- dónde se integran los cambios,
- y cuándo se libera una versión estable.



No se aceptan solo texto: **la representación visual es obligatoria**.

2 Explicación del flujo de trabajo

Acompaña el diagrama con una explicación donde describas, por ejemplo:

- cómo se crea una rama feature,
- cómo se integra en dev,
- cómo se preparan los cambios para el despliegue mensual,
- qué ocurre con main tras el despliegue.

Debe quedar claro **cómo colaboran varios miembros del equipo** sin bloquearse entre ellos.

3 Relación con el despliegue mensual

Incluye explícitamente en el diagrama o en el texto:

- cuándo se decide cerrar una versión,
- qué rama se considera “lista para producción”,
- y por qué este modelo facilita despliegues controlados una vez al mes.

Entregables

- Diagrama de la estrategia Git con:
 - ramas (main, dev, feature/*, bugfix/*) o lo que creas conveniente añadir,
 - merges,
 - punto de despliegue mensual,
 - y **tu nombre visible**.
- Explicación del flujo de trabajo (texto bajo el diagrama o en una página aparte).

Frase sugerida:

“Esta estrategia Git permite a varios equipos trabajar en paralelo y realizar despliegues mensuales de forma controlada.

Nota: Piensa como si fueses el Head de tu equipo y tuvieses que diseñar un sistema de despliegue, como responderías a particularidades concretas de tu equipo en la que se pueden encontrar

■ ENTREGABLE FINAL (PDF)

Donde aparezca cada una de las cosas pedidas en la sección de **cada tarea y una correspondiente explicación** de lo que se muestra en cada captura.

Evaluación

Criterio	Peso
1. Modelado SQL	25%
2. API + Repository + UoW + Tests	25%
3. Seguridad aplicada (JWT + Roles)	25%
4. Git Strategy corporativo	25%

Cada criterio corresponde a un **Resultado de Aprendizaje (RA)** del plan docente.

Cada criterio tiene **4 niveles de logro** equivalentes a la rúbrica institucional.

Cada criterio vale **25% de la actividad**.

■ 1. Modelado SQL (25%)

RA1. Resumir las diferentes tecnologías web y sus interacciones.

IE1. El estudiante identifica las tecnologías implicadas en una aplicación web backend y describe su interacción (API, base de datos y clientes).

Nivel	Qué debe verse
0–4.9	Modelo de datos incompleto o incoherente. No se identifican claramente las



Nivel	Qué debe verse
	entidades principales ni su relación con la API o el sistema backend. No se explica con claridad porque se toman las acciones.
5–6.9	Modelo de datos con algunas tablas relacionadas, pero con relaciones poco claras respecto a su uso dentro de la API.
7–8.9	Modelo de datos coherente con mínimo 4–5 tablas del dominio (Assets, Categories, Hazards, AssetHazardExposure, Conditions o equivalentes) que permiten entender cómo interactúan con la API. Junto con una explicación adecuada.
9–10	Modelo bien estructurado que identifica claramente las entidades del dominio y explica cómo interactúan base de datos, API y cliente dentro de la arquitectura backend. Se pueden observar particularidades en la creación de tablas donde se justifican

■ 2. API + Repository Pattern + Unit of Work + Tests (25%)

RA3. Utilizar técnicas y procedimientos adecuados para el diseño de aplicaciones web.

IE3. El estudiante diseña endpoints y estructuras de acceso a datos coherentes, justificando las decisiones técnicas a partir de documentación.

Nivel	Qué debe verse
0–4.9	No hay endpoints funcionales o las estructuras de acceso a datos son incoherentes. No se justifican decisiones técnicas.
5–6.9	Endpoints básicos. Implementación mínima de Repository o Unit of Work con justificación limitada.
7–8.9	Endpoints funcionales con acceso a datos/memoria mediante Repository Pattern y Unit of Work simples, con explicación adecuada de las decisiones técnicas.
9–10	API bien estructurada con endpoints claros y coherentes, uso adecuado de Repository Pattern y Unit of Work, y justificación técnica basada en documentación o buenas prácticas. Se puede observar un buen uso del lenguaje de programación

■ 3. Seguridad aplicada (25%)

RA2. Interpretar el paradigma cliente–servidor utilizado en tecnologías web.

IE2. El estudiante explica cómo una API responde a peticiones cliente–servidor y lo relaciona con la arquitectura backend.

RA7. Analizar y evaluar información desde fuentes digitales para resolver un problema.

IE7. El estudiante valida el comportamiento de la API mediante pruebas básicas.

Nivel	Qué debe verse
0–4.9	No se presentan pruebas ni evidencias del funcionamiento de la API.
5–6.9	Evidencia parcial del funcionamiento de la API (capturas o pruebas conceptuales), con explicación mínima del flujo cliente–servidor.
7–8.9	Pruebas funcionales o unitarias que validan endpoints principales de la API, mostrando cómo responde la API a peticiones cliente.
9–10	Tests unitarios y/o pruebas funcionales (Postman u otra herramienta) que verifican el comportamiento de la API y documentan claramente el flujo cliente–servidor y los

Nivel	Qué debe verse
	resultados obtenidos. Además se observan particularidades de las peticiones no comentadas en el enunciado.

■ 4. Git Strategy corporativo (25%)

RA4. Analizar la idoneidad de las tecnologías web y su integración según las necesidades del usuario.

IE4. El estudiante argumenta la elección de una estrategia de ramas y ciclo de despliegue en función de los requisitos del equipo.

RA6. Diseñar el flujo de trabajo colaborativo que integra las contribuciones de múltiples desarrolladores de forma controlada.

IE6. El estudiante representa gráficamente y explica un flujo Git profesional con ramas, merges y ciclo de despliegue mensual.

Nivel	Qué debe verse
0–4.9	No se presenta diagrama de ramas ni explicación del flujo de trabajo. No se identifican las ramas principales ni se describe cómo colaboran los desarrolladores. Tampoco se justifica las decisiones tomadas
5–6.9	Diagrama presente y medianamente correcto: falta alguna rama (feature/*, bugfix/*, dev, reléase, etc), los merges están representados correctamente, o la explicación del flujo de trabajo es mínima y menciona el despliegue mensual.
7–8.9	Diagrama correcto con las ramas principales (main, dev, feature/*, bugfix/*), flujo de merges representado y punto de despliegue mensual identificado. La explicación describe cómo colaboran varios desarrolladores sin bloquearse.
9–10	Diagrama completo y claro que representa todas las ramas, los merges entre ellas y el momento del despliegue mensual. La explicación detalla el ciclo completo: creación de feature, integración en dev, preparación del release y despliegue desde main, justificando por qué este modelo facilita el trabajo en equipo y los despliegues controlados.

Fuentes de información

1. Rob, Peter; Coronel, Carlos.

Database Systems: Design, Implementation, and Management. Cengage Learning, 13.^a ed., 2019.

→ Referencia esencial sobre bases de datos relacionales, diseño de tablas, claves, normalización y consultas básicas.

2. Freeman, Adam; Sanderson, Steve.

Pro ASP.NET Core 7: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages. Apress, 2023.

→ Aunque no programen en profundidad, los capítulos iniciales introducen el diseño de endpoints, modelos de datos, controladores y buenas prácticas en servicios web.

3. Hope, Ben; Walberg, Quinn.

ASP.NET Core in Action. Manning Publications, 2023.

→ Excelente para comprender estructuras de API, Minimal APIs, servicios, inyección de dependencias y conceptos de seguridad como JWT o autorización basada en roles.

Bibliografía complementaria

4. Daigneau, Robert.

Service Design Patterns: Fundamental Design Solutions for SOAP/WSDL and RESTful Web Services. Addison-Wesley, 2012.



→ Recurso clave para diseñar endpoints que respondan a preguntas de negocio, desacoplar servicios y estructurar APIs coherentes.

5. Bishop, Matt.

Computer Security: Art and Science. Addison-Wesley, 2019.

→ Introducción rigurosa a autenticación, autorización, control de acceso, roles y políticas de seguridad aplicadas a servicios web.

6. Stallings, William.

Cryptography and Network Security: Principles and Practice. Pearson, 7.^a ed., 2017.

→ Referencia sólida para comprender firmas digitales, tokens, seguridad en APIs y conceptos que justifican la validez del JWT.

Recursos de consulta técnica

7. Microsoft Learn — Documentation for ASP.NET Core and Entity Framework Core

<https://learn.microsoft.com/aspnet/core/>

→ Explicaciones oficiales sobre creación de APIs, modelos, controladores, Minimal API, validación, middleware y autenticación.

8. Microsoft Learn — SQL Server & T-SQL Documentation

<https://learn.microsoft.com/sql/>

→ Primer recurso académico-profesional para consultas SQL, creación de tablas y fundamentos de bases de datos relacionales.

9. Postman Learning Center

<https://learning.postman.com/>

→ Guía para pruebas de endpoints HTTP, manejo de tokens, validación de respuestas y depuración de errores.

Referencias sobre Git y buenas prácticas de versionado

10. Chacon, Scott; Straub, Ben.

Pro Git. Apress, 2.^a ed., 2014.

→ Obra de referencia para commits atómicos, estructura de repositorios, ramas y estrategias de versionado profesional.

11. Loeliger, Jon; McCullough, Matthew.

Version Control with Git. O'Reilly Media, 3.^a ed., 2012.

→ Ideal para comprender flujos colaborativos, documentación del trabajo mediante commits y organización técnica.

